CS 350 — Operating Systems

Vishesh Gupta

University of Waterloo

Contents

1 Introduction **1**
- 1.1 What is an operating system? . . . 1
- 1.2 Layers of a computer system . . . 2
- 1.3 OS abstractions . . . 2
- 1.4 Monolithic kernels vs microkernels . . . 3
- 1.5 OS/161 and System/161 . . . 3
- 1.6 Course roadmap . . . 4

2 Threads and Concurrency **5**
- 2.1 What is a thread? . . . 5
- 2.2 OS/161 thread interface . . . 5
- 2.3 Concurrent execution . . . 6
- 2.4 Context switches . . . 6
- 2.5 Thread states . . . 7
- 2.6 Preemption and timesharing . . . 8
- 2.7 Common pitfalls . . . 8

3 Synchronization **10**
- 3.1 Race conditions and critical sections . . . 10
- 3.2 Achieving mutual exclusion . . . 10
  - 3.2.1 Disabling interrupts . . . 10
  - 3.2.2 Peterson's algorithm (software only) . . . 11
  - 3.2.3 Hardware atomic instructions . . . 11
- 3.3 Spinlocks . . . 11
- 3.4 Locks (mutexes) . . . 12
- 3.5 Semaphores . . . 13
  - 3.5.1 Producer/consumer (bounded buffer) . . . 13
- 3.6 Condition variables . . . 14
- 3.7 Deadlock . . . 15
  - 3.7.1 Prevention . . . 15
  - 3.7.2 Detection and recovery . . . 16

4 Processes, the Kernel, and System Calls **17**
- 4.1 The process abstraction . . . 17
- 4.2 Process management system calls . . . 18
- 4.3 System calls . . . 18
  - 4.3.1 How a system call works on MIPS / OS/161 . . . 19

4.4 Exceptions and interrupts . . . 20
4.4.1 Trapframes vs switchframes . . . 20
4.4.2 Kernel stacks . . . 20
4.5 Implementing fork, waitpid, _exit in OS/161 . . . 21
4.6 MIPS R3000 essentials for OS/161 . . . 22
4.7 Inter-process communication (brief) . . . 22

5 Virtual Memory **24**
5.1 Physical vs virtual addresses . . . 24
5.2 Dynamic relocation (base and bound) . . . 24
5.3 Segmentation . . . 25
5.4 Paging . . . 25
5.5 Multi-level page tables . . . 26
5.6 Translation lookaside buffer (TLB) . . . 26
5.7 OS/161 dumbvm . . . 27
5.8 Where address spaces come from . . . 28
5.9 Paging to secondary storage . . . 28
5.10 Page replacement policies . . . 28
5.10.1 Optimal (MIN / Belady) . . . 29
5.10.2 FIFO . . . 29
5.10.3 LRU (least recently used) . . . 29
5.10.4 Clock (second chance) . . . 29
5.10.5 Working sets and thrashing . . . 29

6 Scheduling **31**
6.1 The scheduling problem . . . 31
6.2 First come, first served (FCFS/FIFO) . . . 31
6.3 Round robin (RR) . . . 32
6.4 Shortest job first (SJF) . . . 32
6.5 Shortest remaining time first (SRTF) . . . 32
6.6 Real threads: CPU bursts and I/O . . . 33
6.7 Multi-level feedback queue (MLFQ) . . . 33
6.8 Linux Completely Fair Scheduler (CFS) . . . 33
6.9 Multicore scheduling . . . 34

7 Devices and I/O **35**
7.1 Devices and buses . . . 35
7.2 Device registers . . . 35
7.3 Polling vs interrupts . . . 36
7.4 Moving data: programmed I/O vs DMA . . . 36
7.5 Hard disk drives . . . 37
7.6 Disk head scheduling . . . 38
7.7 Solid state drives (SSDs) . . . 38
7.8 Other persistent storage . . . 39

8 File Systems **40**
8.1 Files and file systems . . . 40
8.2 File interface . . . 40
8.3 Directories and links . . . 40
8.4 File system implementation . . . 41
8.4.1 Very Simple File System (VSFS) layout . . . 41
8.4.2 I-nodes . . . 42
8.4.3 Directories on disk . . . 42
8.4.4 Example: reading `/foo/bar` . . . 43
8.4.5 Block allocation . . . 43
8.5 Crash consistency . . . 43
8.5.1 fsck . . . 43
8.5.2 Journaling (write-ahead logging) . . . 44
8.6 OS/161 VFS and SFS . . . 44

9 Virtual Machines **45**
9.1 What is a virtual machine? . . . 45
9.2 Types of hypervisors . . . 46
9.3 Virtualizing the CPU . . . 46
9.3.1 Trap and emulate . . . 46
9.4 Virtualizing memory . . . 47
9.5 Virtualizing I/O . . . 47
9.6 Containers (OS-level virtualization) . . . 47
9.7 Networking (brief, for context) . . . 48

Chapter 1

Introduction

Sources. The module list and ordering in these notes follow the archived CS 350 term pages on `student.cs.uwaterloo.ca/~cs350/`, specifically the course outline and reading-materials pages of **Winter 2018** (modules: Introduction, Threads, Synchronization, Processes/Kernel/System Calls, Virtual Memory, Scheduling, Devices and I/O, File Systems; the "Interprocess Communication and Networking" module was listed as *not covered*) and **Fall 2020** (same modules, with a new *Virtual Machines* module at the end). The recommended (optional) textbook in both terms was Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP); older terms used Silberschatz, Galvin and Gagne. Assignments were done in OS/161 on the System/161 MIPS simulator.

1.1 What is an operating system?

There is no single crisp definition; it helps to look at the OS from three viewpoints.

Application view The OS provides an *execution environment* for running programs. It gives each program the resources it needs (CPU time, memory, storage) and an *interface* to those resources that is more convenient than the bare hardware: files instead of disk sectors, virtual address spaces instead of physical RAM, threads instead of a single CPU.

System view The OS is a *resource manager*. It allocates resources among many concurrently running programs, isolates them from each other (one buggy program should not crash another), and enforces policy (who may access which file, who runs next).

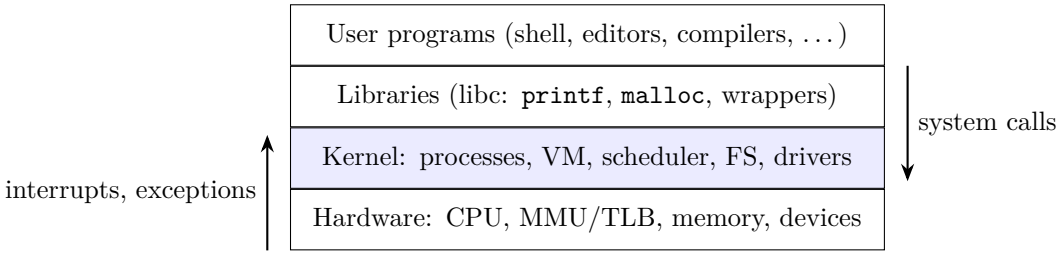
Implementation view The OS is a *concurrent, real-time program*. Concurrency is inherent: many programs, devices and interrupts are all active at once. Real-time because hardware events (a network packet, a disk completion) must be handled promptly.

Definition 1.1 (Kernel)**.** The **kernel** is the part of the OS that runs in privileged (kernel / supervisor) mode and responds to system calls, interrupts and exceptions. The **operating system** as a whole also includes user-level parts: system libraries, daemons/servers, shells, utilities.

Privilege

Hardware supports at least two execution modes. In *kernel mode* the CPU may execute privileged instructions (halt, change the page table base / TLB, disable interrupts, talk to devices). In *user mode* such instructions trap. Applications run in user mode, so the only way for an application to use hardware is to ask the kernel through a *system call*.

1.2 Layers of a computer system



The kernel sits between the user programs and the hardware. Control enters the kernel in exactly three ways:

1. **System calls** — deliberate requests from a user program (via a trap instruction such as `syscall` on MIPS).
2. **Exceptions** — errors or conditions caused by the running instruction (divide by zero, illegal address, TLB miss).
3. **Interrupts** — asynchronous signals from devices (timer tick, disk done).

After handling, the kernel either returns to the interrupted program or switches to another.

1.3 OS abstractions

A large part of the course is about how the OS implements a few key abstractions.

Abstraction	Hides / virtualizes	Module
Thread	the CPU (sequential execution)	Threads, Scheduling
Process	address space + threads + resources	Processes
Address space	physical memory	Virtual memory
File / directory	disk blocks	File systems
Device driver interface	device registers	Devices and I/O
Virtual machine	the entire hardware	Virtual machines

OSTEP organizes the same material as three "easy pieces": *virtualization* (CPU and memory), *concurrency* (threads, locks, condition variables), and *persistence* (devices and file systems). CS 350 covers all three, with concurrency first because writing a kernel requires synchronization from day one.

1.4 Monolithic kernels vs microkernels

Monolithic kernel Everything (file systems, drivers, networking, VM, scheduling) runs in kernel mode in one address space. Fast (procedure calls between subsystems) but a bug anywhere can crash the system. Linux, traditional Unix, OS/161.

Microkernel Only a minimum runs in kernel mode (address spaces, threads, IPC); file systems, drivers, etc. run as user-level server processes. More robust and modular, but more IPC overhead. Mach, L4, QNX, MINIX 3.

Hybrid Mixtures, e.g. Windows NT and macOS (XNU = Mach + BSD).

1.5 OS/161 and System/161

The assignments use **OS/161**, a small teaching operating system originally from Harvard, which runs on **System/161**, a simulator of a MIPS R3000 based machine (`sys161`).

- The kernel is written in C (with a little MIPS assembly for low-level entry/exit and context switching).
- Source tree highlights: `kern/thread` (threads, scheduler, synchronization primitives in `synch.c`), `kern/arch/mips` (trap handling, `switchframe`, TLB code), `kern/syscall`, `kern/vm` (initially the very simple `dumbvm`), `kern/vfs` and `kern/fs` (SFS), plus user-level programs and `libc` under `user/`.
- Debugging is done by attaching `cs350-gdb` to the simulator (`sys161 -w`).

Typical assignment progression (2016–2021):

A0 Getting familiar with the code base, building and running the kernel, `gdb`.

A1 Implementing **locks** and **condition variables**, then solving a synchronization problem (e.g. a traffic intersection or cat/mouse simulation).

A2a Process system calls: `fork`, `getpid`, `waitpid`, `_exit`.

A2b `execv` with argument passing.

A3 Virtual memory: TLB replacement, read-only text segment, replacing `dumbvm` with a page-granular physical memory allocator (coremap).

```
# build and run OS/161 (typical workflow)
cd os161-1.99/kern/compile/ASST2
bmake depend && bmake && bmake install
cd ~/cs350-os161/root
sys161 kernel-ASST2          # boots to the OS/161 menu
sys161 -w kernel-ASST2       # wait for debugger
cs350-gdb kernel             # in another terminal: target remote unix:.sockets/gdb
```

1.6 Course roadmap

1. Threads and concurrency — how one CPU appears to run many threads.
2. Synchronization — making concurrent threads cooperate safely.
3. Processes and the kernel — system calls, exceptions, interrupts, user/kernel boundary.
4. Virtual memory — address translation, paging, TLBs, page replacement.
5. Scheduling — choosing which thread runs next.
6. Devices and I/O — talking to hardware, disks and SSDs.
7. File systems — persistent storage, i-nodes, crash consistency.
8. Virtual machines — running whole operating systems on top of a hypervisor.

Summary

- An OS provides an execution environment (abstractions + interface), manages and isolates resources, and is itself a concurrent real-time program.
- The kernel runs in privileged mode; it is entered only through system calls, exceptions and interrupts.
- Key abstractions: threads, processes, address spaces, files, virtual machines.
- CS 350 assignments build process management and virtual memory into OS/161, a small monolithic kernel running on a simulated MIPS R3000.

Chapter 2

Threads and Concurrency

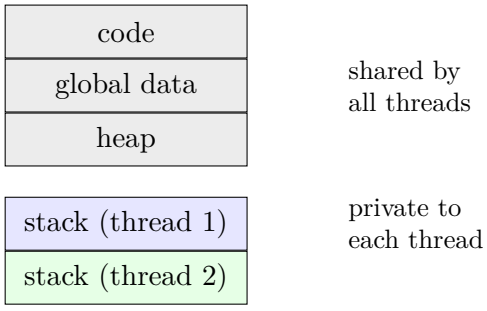
2.1 What is a thread?

Definition 2.1 (Thread)**.** A **thread** is a sequence of instructions being executed, together with the state needed to continue executing it: a program counter, register values, and a stack. A **sequential** program has one thread; a **concurrent** program has several threads that may run at the same time.

Why use threads?

- **Resource utilization**: when one thread blocks (e.g. waiting for disk I/O), another can use the CPU.
- **Parallelism**: on a multicore machine, threads run simultaneously and finish the work faster.
- **Responsiveness**: a GUI thread keeps responding while a worker thread computes.
- **Program structure**: a server handling many clients is naturally written as one thread per client.

Threads in the same process *share* the address space: code, global variables and the heap. Each thread has its *own* stack, program counter and registers.



2.2 OS/161 thread interface

```
/* kern/include/thread.h */
int  thread_fork(const char *name,          /* name for debugging */
```

```
                struct proc *proc,         /* process the thread joins (NULL = current)
    */
                void (*func)(void *, unsigned long), /* entry point */
                void *data1, unsigned long data2);   /* args to func */
void thread_exit(void);   /* terminate the calling thread (never returns) */
void thread_yield(void);  /* voluntarily give up the CPU; stays runnable */
```

Example 2.2 (Forking worker threads)**.**

```
static volatile int total = 0;    /* shared! needs synchronization (Ch. 3) */

static void worker(void *p, unsigned long n) {
    (void)p;
    kprintf("worker %lu running\n", n);
    thread_exit();
}

int run_workers(void) {
    for (unsigned long i = 0; i < 4; i++) {
        int err = thread_fork("worker", NULL, worker, NULL, i);
        if (err) panic("thread_fork failed: %s\n", strerror(err));
    }
    return 0;
}
```

The order in which the workers print is *non-deterministic*: it depends on the scheduler.

OS/161 has no `thread_join`; the analogous POSIX API is `pthread_create`, `pthread_join`, `pthread_exit`. Joining is something you build yourself with semaphores or condition variables.

2.3 Concurrent execution

On a single core, threads take turns. Two models of concurrency:

Multiprocessing / multicore P processors each run one thread at a time; up to P threads truly run in parallel.

Timesharing Multiple threads share one processor by rapidly switching between them.

Real systems combine both: each core is timeshared.

2.4 Context switches

Definition 2.3 (Context switch)**.** A **context switch** is the act of pausing the currently running thread (saving its register state), choosing another thread to run (the *scheduler*), and resuming that thread (restoring its register state).

The saved state of a thread is called its **context**. In OS/161 the context is saved on the thread's own stack in a `switchframe`, and the thread structure remembers the stack pointer.

Context switch in OS/161

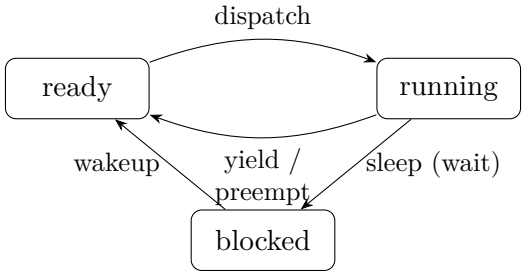
1. The running thread calls `thread_yield()` (or is preempted, or blocks).
2. `thread_yield` calls `thread_switch(S_READY, ...)`, which puts the current thread on the ready queue and asks the scheduler for the next thread.
3. `thread_switch` calls the assembly routine `switchframe_switch`, which
   - pushes the *callee-saved* registers `s0`–`s8`, `gp`, `ra` onto the current stack (only these, since caller-saved registers were saved by the normal C calling convention before calling `thread_switch`),
   - saves the stack pointer into the old thread's structure,
   - loads the new thread's saved stack pointer,
   - pops the new thread's registers and returns — *into the new thread*.

```
# simplified switchframe_switch(a0 = &old->t_context, a1 = &new->t_context)
switchframe_switch:
    addi sp, sp, -40        # allocate stack space for 10 registers
    sw   ra, 36(sp)
    sw   gp, 32(sp)
    sw   s8, 28(sp)
    sw   s6, 24(sp)
    # ... s5..s0 ...
    sw   sp, 0(a0)          # old thread: remember its stack pointer
    lw   sp, 0(a1)          # new thread: switch to its stack
    nop                     # load delay slot
    lw   s0, 0(sp)
    # ... restore s1..s8, gp, ra ...
    j    ra                 # return into the NEW thread
    addi sp, sp, 40         # (delay slot) pop the frame
```

Why only callee-saved registers?

`switchframe_switch` is called like a normal function. By the MIPS calling convention the caller (`thread_switch`) has already saved any caller-saved (temporary) registers it needs. So the switch code only has to preserve the callee-saved ones. This is different from an interrupt (Ch. 4), where *all* registers must be saved in a trapframe because the interrupted code did not expect to be interrupted.

2.5 Thread states



Running currently executing on a CPU.

Ready could run, waiting for a CPU; kept on the **ready queue**.

Blocked waiting for something (a lock, I/O, a semaphore); kept on a **wait channel** (`struct wchan` in OS/161). Only another thread or an interrupt handler can wake it, moving it back to ready.

In OS/161 a thread blocks with `wchan_sleep(wc, lock)` and is woken with `wchan_wakeone` / `wchan_wakeall`. Transitions out of running are: `thread_yield` (running → ready), `wchan_sleep` (running → blocked), `thread_exit` (running → zombie/destroyed).

2.6 Preemption and timesharing

If a thread never calls `thread_yield` or blocks, a cooperative system would let it hog the CPU forever. **Preemption** forces a context switch.

Definition 2.4 (Preemptive scheduling)**.** The kernel uses a periodic hardware **timer interrupt** to regain control. Each thread is allowed to run for at most a **scheduling quantum** (time slice). When the timer interrupt handler notices that the running thread's quantum has expired, it calls `thread_yield` on the thread's behalf.

The sequence on a preemption in OS/161:

1. Timer interrupt fires; the CPU jumps to the exception handler, which saves the *entire* register state in a `trapframe` on the thread's kernel stack.
2. `mainbus_interrupt` → `hardclock()`; if the quantum is used up, `hardclock` calls `thread_yield()`.
3. `thread_yield` does a normal context switch (pushing a `switchframe`).

So the stack of a preempted thread contains (from bottom to top): its normal frames, a `trapframe`, frames for the interrupt handling functions, `thread_yield`'s frame, and finally a `switchframe`. When the thread is later resumed, it unwinds in reverse order and returns from the interrupt exactly where it left off.

Example 2.5 (Round-robin timesharing)**.** Three CPU-bound threads A, B, C with quantum q on one core run as $A, B, C, A, B, C, \ldots$, each for q time units. Smaller q gives better responsiveness but more context-switch overhead (each switch costs saving/restoring registers plus cache/TLB effects). Typical quanta are a few milliseconds.

2.7 Common pitfalls

- Assuming threads run in fork order — they may run in any interleaving.
- Passing a pointer to a stack variable of the parent to a child thread, then returning from the parent function: the child now reads garbage.
- Confusing `thread_yield` (stays runnable) with sleeping on a wait channel (needs a wakeup).

Summary

- A thread = PC + registers + stack; threads of a process share code, data and heap.
- A context switch saves the old thread's registers (OS/161: a `switchframe` on its stack), picks a new thread, and restores its registers.
- Threads are running, ready (ready queue) or blocked (wait channel).
- Preemption uses timer interrupts and a scheduling quantum to timeshare the CPU.

Chapter 3

Synchronization

3.1 Race conditions and critical sections

Threads in a process share global data. Without coordination, concurrent access gives results that depend on timing.

Example 3.1 (Lost update)**.** Two threads each run `total++` 1000 times on a shared `int total = 0`. The statement compiles to roughly three instructions:

```
lw   t0, total      # load
addi t0, t0, 1      # increment
sw   t0, total      # store
```

Interleaving: T1 loads 0, T2 loads 0, T1 stores 1, T2 stores 1. One increment is lost. The final value can be anything from 2 to 2000 (2 needs a tricky interleaving, but is possible).

Definition 3.2. A **race condition** occurs when the result of a program depends on the relative timing or interleaving of threads. A **critical section** is a piece of code that accesses shared data and must not be executed by more than one thread at a time. **Mutual exclusion** is the property that at most one thread is inside the critical section.

Requirements for a correct critical section solution:

1. **Mutual exclusion**: at most one thread inside.
2. **Progress**: if no thread is inside and some want in, one of them eventually gets in.
3. **Bounded waiting** (fairness): a thread that wants in is not overtaken forever.

Compiler and CPU reordering. The compiler may keep a shared variable in a register or reorder memory accesses; `volatile` forces every access to go to memory but does *not* make operations atomic. Modern CPUs also reorder loads/stores; synchronization primitives contain *memory barriers* (`sync` on MIPS) so that accesses do not leak out of critical sections.

3.2 Achieving mutual exclusion

3.2.1 Disabling interrupts

On a *single* processor, a thread can only lose the CPU through an interrupt. Disabling interrupts (`splhigh()` in OS/161) around a critical section therefore gives mutual exclusion. Drawbacks:

only the kernel may do it; it does not work on multiprocessors; it delays device handling, so critical sections must be very short.

3.2.2 Peterson's algorithm (software only)

For two threads, using only loads and stores (assuming sequential consistency):

```
volatile bool flag[2] = {false, false};
volatile int turn;

void enter(int i) {          /* i is 0 or 1 */
    int j = 1 - i;
    flag[i] = true;          /* I want in */
    turn = j;                /* but you go first */
    while (flag[j] && turn == j) { /* busy wait */ }
}
void leave(int i) { flag[i] = false; }
```

Correct but impractical: two threads only, busy waits, and breaks under memory reordering.

3.2.3 Hardware atomic instructions

Definition 3.3 (Test-and-set)**.** `TestAndSet(addr, v)` atomically stores `v` at `addr` and returns the old value.

x86 provides `xchg`; many architectures provide `compare-and-swap`. MIPS instead has a pair: `ll` (load-linked) and `sc` (store-conditional). `sc` only succeeds if no other write to that address happened since the matching `ll`; it returns success/failure.

```
/* test-and-set built from ll/sc, as in OS/161's spinlock_data_testandset */
static inline int test_and_set(volatile int *addr) {
    int x, y;
    y = 1;
    __asm volatile(
        ".set push; .set mips32;"
        "ll %0, 0(%2);"       /* x = *addr (linked) */
        "sc %1, 0(%2);"       /* try *addr = 1; y = success? */
        ".set pop"
        : "=&r"(x), "+r"(y) : "r"(addr));
    if (y == 0) return 1;     /* sc failed: pretend lock was held */
    return x;                 /* old value */
}
```

3.3 Spinlocks

```
void spinlock_acquire(struct spinlock *lk) {
    splraise(IPL_NONE, IPL_HIGH);      /* disable interrupts on this CPU */
    for (;;) {
        if (lk->lk_lock != 0) continue;            /* test first (cheap read) */
        if (test_and_set(&lk->lk_lock) == 0) break; /* got it */
    }
    lk->lk_holder = curcpu->c_self;
```

```
    membar_any_any();
}
void spinlock_release(struct spinlock *lk) {
    membar_any_any();
    lk->lk_holder = NULL;
    lk->lk_lock = 0;
    spllower(IPL_HIGH, IPL_NONE);
}
```

- A spinlock *busy-waits*: the waiting thread burns CPU.
- OS/161 spinlocks also disable interrupts on the local CPU, so the holder cannot be preempted (which would make others spin for a whole quantum) and an interrupt handler on the same CPU cannot deadlock trying to take the same lock.
- Use spinlocks only for very short critical sections. Never block (sleep) while holding a spinlock.

3.4 Locks (mutexes)

A **lock** provides mutual exclusion like a spinlock, but a thread that cannot acquire it *blocks* instead of spinning. Locks also have an **owner**: only the thread that acquired a lock may release it.

```
struct lock {
    char *lk_name;
    struct wchan *lk_wchan;
    struct spinlock lk_spin;   /* protects the fields below */
    struct thread *lk_owner;   /* NULL if free */
};

void lock_acquire(struct lock *lk) {
    spinlock_acquire(&lk->lk_spin);
    while (lk->lk_owner != NULL) {
        /* atomically releases lk_spin and sleeps; reacquires on wakeup */
        wchan_sleep(lk->lk_wchan, &lk->lk_spin);
    }
    lk->lk_owner = curthread;
    spinlock_release(&lk->lk_spin);
}

void lock_release(struct lock *lk) {
    KASSERT(lock_do_i_hold(lk));
    spinlock_acquire(&lk->lk_spin);
    lk->lk_owner = NULL;
    wchan_wakeone(lk->lk_wchan, &lk->lk_spin);
    spinlock_release(&lk->lk_spin);
}
```

Why `while` and not `if`?

After being woken, another thread may grab the lock before the woken thread runs. The woken thread must re-check the condition. Also, `wchan_sleep` must release the spinlock and sleep *atomically*, otherwise a wakeup could happen between the release and the sleep and be lost.

	Spinlock	Lock
Waiting	busy-wait	block on wait channel
Good for	very short sections, interrupt handlers	longer sections
Can sleep while holding?	no	yes
Overhead	low when uncontended	context switch when contended

3.5 Semaphores

Definition 3.4 (Semaphore (Dijkstra))**.** A **semaphore** is an object with a non-negative integer value and two atomic operations:

- `P` (wait, "proberen"): if the value is 0, block until it is positive; then decrement it.
- `V` (signal, "verhogen"): increment the value (waking a waiter if any).

```
void P(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
    while (sem->sem_count == 0)
        wchan_sleep(sem->sem_wchan, &sem->sem_lock);
    sem->sem_count--;
    spinlock_release(&sem->sem_lock);
}
void V(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
    sem->sem_count++;
    wchan_wakeone(sem->sem_wchan, &sem->sem_lock);
    spinlock_release(&sem->sem_lock);
}
```

Kinds of use:

- **Binary semaphore** (initial value 1): like a lock, but no owner — any thread can `V`.
- **Counting semaphore** (initial value N): N identical resources.
- **Signalling** (initial value 0): one thread waits (`P`) for an event that another thread announces (`V`). E.g. a parent waits for k children by calling `P` k times; each child calls `V` once when done.

3.5.1 Producer/consumer (bounded buffer)

```
#define N 16
item buffer[N]; int in = 0, out = 0;
struct semaphore *items;   /* initial 0: number of full slots */
struct semaphore *spaces;  /* initial N: number of empty slots */
```

```
struct lock *mutex;        /* protects buffer, in, out */

void produce(item x) {
    P(spaces);                    /* wait for an empty slot */
    lock_acquire(mutex);
    buffer[in] = x; in = (in + 1) % N;
    lock_release(mutex);
    V(items);                     /* announce a full slot */
}
item consume(void) {
    P(items);
    lock_acquire(mutex);
    item x = buffer[out]; out = (out + 1) % N;
    lock_release(mutex);
    V(spaces);
    return x;
}
```

Remark. Swapping the order (taking `mutex` before `P(spaces)`) can deadlock: a producer holding the mutex sleeps on a full buffer, and no consumer can get the mutex to free a slot.

3.6 Condition variables

A **condition variable** lets a thread wait, *inside* a critical section protected by a lock, until some condition on shared data becomes true. It is always used together with a lock.

`cv_wait(cv, lk)` atomically release `lk` and block on `cv`; when woken, reacquire `lk` before returning.

`cv_signal(cv, lk)` wake one thread waiting on `cv` (caller holds `lk`).

`cv_broadcast(cv, lk)` wake all threads waiting on `cv`.

```
void cv_wait(struct cv *cv, struct lock *lk) {
    spinlock_acquire(&cv->cv_spin);
    lock_release(lk);                          /* release only after holding cv_spin */
    wchan_sleep(cv->cv_wchan, &cv->cv_spin);  /* so no signal can be lost */
    spinlock_release(&cv->cv_spin);
    lock_acquire(lk);
}
```

OS/161 (like POSIX) uses **Mesa semantics**: a signalled thread is merely made ready; by the time it runs the condition may be false again. Hence the golden rule:

Always wait in a loop

```
lock_acquire(lk);
while (!condition)        /* NOT "if" */
    cv_wait(cv, lk);
/* condition holds and we hold lk */
lock_release(lk);
```

(Hoare semantics, where the signaller hands the lock directly to the waiter, would allow `if`, but almost no real system implements it.)

Example 3.5 (Bounded buffer with a lock and two CVs)**.**

```
int count = 0;
struct lock *mtx; struct cv *notfull, *notempty;

void produce(item x) {
    lock_acquire(mtx);
    while (count == N) cv_wait(notfull, mtx);
    buffer[in] = x; in = (in + 1) % N; count++;
    cv_signal(notempty, mtx);
    lock_release(mtx);
}
item consume(void) {
    lock_acquire(mtx);
    while (count == 0) cv_wait(notempty, mtx);
    item x = buffer[out]; out = (out + 1) % N; count--;
    cv_signal(notfull, mtx);
    lock_release(mtx);
    return x;
}
```

3.7 Deadlock

Example 3.6. Thread 1: `lock_acquire(A); lock_acquire(B);` Thread 2: `lock_acquire(B); lock_acquire(A);`. If each gets its first lock, both wait forever.

Definition 3.7 (Deadlock)**.** A set of threads is **deadlocked** if each is blocked waiting for a resource held by another thread in the set, so none can make progress.

Theorem 3.8 (Coffman conditions)**.** *Deadlock can occur only if all four hold simultaneously:*

1. ***Mutual exclusion*** *(no sharing): a resource can be held by only one thread.*
2. ***Hold and wait****: a thread holds resources while requesting more.*
3. ***No preemption****: resources cannot be forcibly taken away.*
4. ***Circular wait****: a cycle of threads each waiting for the next.*

3.7.1 Prevention

Break one condition:

- *No hold and wait*: request all resources at once, or release everything before requesting more (wasteful, may starve).
- *Preemption*: if a request cannot be satisfied, release what you hold and retry (e.g. `lock_tryacquire` then back off). Risk of livelock.
- *Resource ordering*: number all resources and always acquire in increasing order. Then a cycle is impossible: in a cycle some thread would hold a higher-numbered resource while waiting for a lower one. This is the most common practical technique.

3.7.2 Detection and recovery

Model the system as a **resource allocation graph**: an edge $T \to R$ if T requests R, $R \to T$ if R is allocated to T. With single-instance resources, deadlock $\iff$ a cycle exists. With multiple instances, use a detection algorithm:

```
/* U[i][j]: units of resource j requested by thread i
   A[i][j]: units of resource j allocated to thread i
   avail[j]: free units of resource j */
bool done[n] = {false};
bool progress = true;
while (progress) {
    progress = false;
    for (i = 0; i < n; i++)
        if (!done[i] && U[i] <= avail) {   /* componentwise compare */
            avail += A[i];                 /* pretend T_i finishes, frees all */
            done[i] = true; progress = true;
        }
}
/* every thread with done[i] == false is deadlocked */
```

Example 3.9. Resources R_1, R_2 with 1 and 2 units. $A = \{T_1 : (1,0), T_2 : (0,1), T_3 : (0,1)\}$, so avail $= (0,0)$. Requests $U = \{T_1 : (0,1), T_2 : (1,0), T_3 : (0,0)\}$. T_3 can finish (needs nothing): avail $= (0,1)$. Now T_1 can finish: avail $= (1,1)$. Then T_2. No deadlock. If instead T_3 also requested $(1,0)$, nobody could proceed: T_1, T_2, T_3 deadlocked.

Recovery: kill a thread in the cycle, or preempt and roll back. Many systems simply ignore deadlock (the "ostrich algorithm") and rely on lock-ordering discipline in code.

Summary

- Races arise when interleavings of accesses to shared data produce different results; critical sections must be mutually exclusive.
- Hardware atomics (test-and-set, CAS, `ll`/`sc`) build spinlocks; spinlocks plus wait channels build blocking locks, semaphores and condition variables.
- Semaphores: `P`/`V` on a counter; good for counting resources and signalling.
- CVs: wait for a condition while holding a lock; Mesa semantics $\Rightarrow$ wait in a `while` loop.
- Deadlock needs mutual exclusion, hold-and-wait, no preemption, circular wait; prevent by lock ordering, or detect with a graph / matrix algorithm.

Chapter 4

Processes, the Kernel, and System Calls

4.1 The process abstraction

Definition 4.1 (Process)**.** A **process** is an environment in which an application program runs. It consists of

- one or more **threads**;
- a virtual memory **address space** (code, data, heap, stacks);
- other resources: open file descriptors, current working directory, process ID, parent/child relationships, credentials.

Processes are **isolated** from each other: one process cannot read another's memory or files descriptors unless the kernel explicitly allows sharing. The kernel keeps per-process state in a process control block (`struct proc` in OS/161).

```
struct proc {                       /* kern/include/proc.h (with A2 additions) */
    char *p_name;
    struct spinlock p_lock;
    struct threadarray p_threads;   /* threads in this process */
    struct addrspace *p_addrspace;  /* virtual address space */
    struct vnode *p_cwd;            /* current directory */
    /* added for assignment 2: */
    pid_t p_pid;
    struct proc *p_parent;
    int p_exitcode; bool p_exited;
    struct cv *p_waitcv;
};
```

4.2 Process management system calls

Operation	Linux / POSIX	OS/161
Create	`fork`, `clone`	`fork`
Replace program	`execve`	`execv`
Terminate	`_exit`, `exit`	`_exit`
Wait for child	`waitpid`, `wait`	`waitpid`
Identity	`getpid`, `getppid`	`getpid`

`fork()` creates a new process (the *child*) that is a *clone* of the caller (the *parent*): a copy of the address space, same open files, one thread that starts executing by returning from `fork`. Returns the child's PID in the parent and 0 in the child.

`_exit(code)` terminates the calling process. Its exit status is kept until the parent collects it; a terminated-but-uncollected process is a **zombie**.

`waitpid(pid, &status, options)` blocks until child `pid` has exited and returns its exit status. A process may only wait for its own children.

`execv(path, argv)` replaces the calling process's address space with a new program loaded from `path`, passing arguments `argv`. The PID, open files, and parent/child relationships stay the same. On success it does not return.

Example 4.2 (The classic fork/exec/wait pattern).

```c
int main(void) {
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); _exit(1); }
    if (pid == 0) {                          /* child */
        char *args[] = { "/bin/ls", "-l", NULL };
        execv(args[0], args);
        _exit(127);                          /* only reached if execv failed */
    }
    int status;                              /* parent */
    waitpid(pid, &status, 0);
    printf("child exited with %d\n", WEXITSTATUS(status));
    return 0;
}
```

Example 4.3 (Counting forks). `for (i = 0; i < 3; i++) fork();` creates $2^3 = 8$ processes in total (7 new), since every existing process doubles at each iteration.

4.3 System calls

A system call is the interface between user programs and the kernel. User code cannot call kernel functions directly (different privilege level, kernel memory is protected), so it executes a special trap instruction.

4.3.1 How a system call works on MIPS / OS/161

1. The application calls a library wrapper, e.g. `fork()` in `libc`.
2. The wrapper puts the **system call code** in register `v0` (e.g. `SYS_fork`), arguments in `a0`–`a3` (more on the user stack), and executes `syscall`.
3. `syscall` raises an exception: the CPU switches to kernel mode, records the cause (`EX_SYS`) and the faulting PC in coprocessor 0 registers, and jumps to the fixed exception handler address.
4. The kernel saves the user's registers in a **trapframe** on the thread's kernel stack and calls `mips_trap` → `syscall(tf)`, which dispatches on `tf->tf_v0`.
5. The result is placed in the trapframe: `v0` = return value (or error code), `a3` = 0 for success / 1 for failure. The PC in the trapframe is advanced by 4 so the `syscall` instruction is not re-executed.
6. The kernel restores registers from the trapframe and executes `rfe`/`jr` to return to user mode. The libc wrapper checks `a3`; on failure it stores `v0` into `errno` and returns −1.

```
# user-level stub generated for each system call (userland/lib/libc/syscalls.S)
reboot:
    j    __syscall
    addiu v0, $0, SYS_reboot     # delay slot: syscall number in v0
__syscall:
    syscall                      # trap into the kernel
    beq  a3, $0, 1f              # a3 == 0 -> success
    nop
    sw   v0, errno               # failure: set errno
    li   v1, -1
    li   v0, -1                  # return -1
1:  j    ra
    nop
```

```
/* kern/arch/mips/syscall/syscall.c (abbreviated) */
void syscall(struct trapframe *tf) {
    int callno = tf->tf_v0, err;
    int32_t retval = 0;
    switch (callno) {
    case SYS_getpid:  err = sys_getpid((pid_t *)&retval); break;
    case SYS_fork:    err = sys_fork(tf, (pid_t *)&retval); break;
    case SYS__exit:   sys__exit((int)tf->tf_a0); panic("unreachable");
    case SYS_waitpid: err = sys_waitpid((pid_t)tf->tf_a0, (userptr_t)tf->tf_a1,
                                        (int)tf->tf_a2, (pid_t *)&retval); break;
    default:          err = ENOSYS; break;
    }
    if (err) { tf->tf_v0 = err;    tf->tf_a3 = 1; }
    else     { tf->tf_v0 = retval; tf->tf_a3 = 0; }
    tf->tf_epc += 4;              /* skip over the syscall instruction */
}
```

Never trust user pointers

A pointer passed as a system call argument (e.g. `status` in `waitpid`) is a *user* virtual address. It might be NULL, point into the kernel, or be unmapped. The kernel must use `copyin`/`copyout` (and `copyinstr`), which validate the address and handle faults gracefully, returning `EFAULT` instead of crashing the kernel.

4.4 Exceptions and interrupts

Definition 4.4. An **exception** is a synchronous event caused by the current instruction (system call, arithmetic overflow, illegal instruction, address error, TLB miss/fault). An **interrupt** is an asynchronous event raised by a device (timer, disk, network, serial console).

On MIPS R3000 both are handled the same way: the processor switches to kernel mode, disables interrupts, saves the PC in `EPC`, records the reason in the `Cause` register, and jumps to a fixed address: `0x80000000` for user-mode TLB misses (UTLB), `0x80000080` for everything else. OS/161 exception codes include:

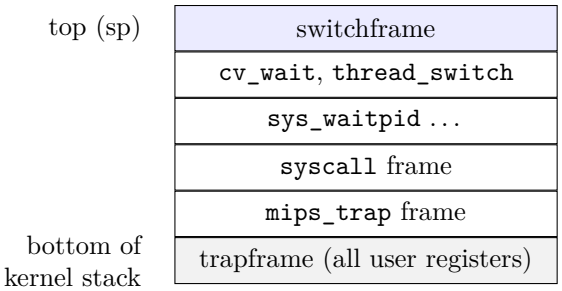
Code	Name	Meaning
0	`EX_IRQ`	interrupt
1	`EX_MOD`	write to a read-only TLB page
2, 3	`EX_TLBL`, `EX_TLBS`	TLB miss on load / store
4, 5	`EX_ADEL`, `EX_ADES`	address error on load / store
8	`EX_SYS`	system call
10	`EX_RI`	reserved (illegal) instruction
12	`EX_OVF`	arithmetic overflow

4.4.1 Trapframes vs switchframes

- The **trapframe** is built by the exception handler (`common_exception`) and saves *all* general registers plus `EPC`, `Cause`, `Status`, `hi`/`lo`, because the interrupted code had no warning.
- The **switchframe** is built by `switchframe_switch` during a context switch and holds only callee-saved registers (Ch. 2).

4.4.2 Kernel stacks

Each thread has *two* stacks: a user stack (in the process's address space, used while running user code) and a **kernel stack** (in kernel memory, used while running in the kernel on that thread's behalf). On an exception the handler switches `sp` to the thread's kernel stack before saving anything. Reasons: the user stack pointer might be invalid or malicious, and kernel data must not be visible to user code.



Kernel stack of a thread that made a `waitpid` system call and blocked.

4.5 Implementing fork, waitpid, _exit in OS/161

sys_fork

1. Create a new `struct proc` (`proc_create_runprogram`), assign a PID.
2. Copy the parent's address space with `as_copy`.
3. Record parent/child relationship.
4. Copy the parent's trapframe to the heap (the parent's kernel stack may change before the child runs).
5. `thread_fork` a child thread whose entry function (`enter_forked_process`) copies the trapframe onto its own kernel stack, sets `v0 = 0`, `a3 = 0`, `epc += 4`, activates the address space, and calls `mips_usermode` to return to user mode.
6. Parent returns the child's PID.

sys_waitpid and sys__exit

```
void sys__exit(int exitcode) {
    struct proc *p = curproc;
    lock_acquire(proc_table_lock);
    p->p_exitcode = _MKWAIT_EXIT(exitcode);
    p->p_exited = true;
    cv_broadcast(p->p_waitcv, proc_table_lock);   /* wake a waiting parent */
    /* children of p become orphans; destroy those already exited */
    lock_release(proc_table_lock);
    as_deactivate();
    as_destroy(proc_getas(p));                    /* free memory now */
    proc_remthread(curthread);
    /* if parent already exited, nobody can wait: destroy p entirely */
    thread_exit();
}

int sys_waitpid(pid_t pid, userptr_t status, int options, pid_t *retval) {
    struct proc *child = proc_lookup(pid);
    if (options != 0) return EINVAL;
    if (child == NULL) return ESRCH;
    if (child->p_parent != curproc) return ECHILD;
```

```
    lock_acquire(proc_table_lock);
    while (!child->p_exited)
        cv_wait(child->p_waitcv, proc_table_lock);
    int code = child->p_exitcode;
    lock_release(proc_table_lock);
    int err = copyout(&code, status, sizeof(int));
    if (err) return err;
    *retval = pid;
    return 0;
}
```

sys_execv

1. Copy the program path and the argument strings from the *old* address space into kernel memory (`copyinstr`) — the old address space is about to vanish.
2. Open the executable (`vfs_open`), create a new address space, activate it, and load the ELF file (`load_elf`).
3. Define the user stack (`as_define_stack`); copy the argument strings and then the `argv` pointer array onto the new stack, keeping pointers 4-byte aligned (8-byte aligned stack top), with `argv[argc] = NULL`.
4. Destroy the old address space; call `enter_new_process(argc, argv_uaddr, sp, entrypoint)`, which never returns.

4.6 MIPS R3000 essentials for OS/161

- 32 general registers: `zero`, `at`, `v0-v1` (return values / syscall number), `a0-a3` (arguments), `t0-t9` (caller-saved), `s0-s8` (callee-saved), `k0-k1` (reserved for the kernel's exception handler), `gp`, `sp`, `ra`.
- Coprocessor 0 holds system registers: `Status` (interrupt enable, user/kernel mode bits in a small stack), `Cause`, `EPC`, `BadVAddr`, TLB registers (`EntryHi`, `EntryLo`, `Index`).
- Branch and jump delay slots, and a load delay slot: the instruction after a branch always executes.
- 32-bit virtual addresses split into segments: `kuseg` (0x00000000–0x7fffffff, user, TLB-mapped), `kseg0` (0x80000000–0x9fffffff, kernel, cached, direct-mapped to physical by subtracting 0x80000000), `kseg1` (0xa0000000–0xbfffffff, kernel, uncached, direct-mapped; used for devices), `kseg2` (kernel, TLB-mapped, unused in OS/161).

4.7 Inter-process communication (brief)

Processes are isolated, so cooperation needs kernel-provided mechanisms: pipes, files, signals, shared memory regions, message queues, and sockets (which also work across machines). In W18 this material was listed as not covered; it appears here only for context.

Summary

- A process = threads + address space + resources, isolated by the kernel.
- `fork` clones, `execv` replaces the program, `_exit` terminates (zombie until collected), `waitpid` collects a child's status.
- System calls enter the kernel via a trap; the kernel saves a trapframe on the thread's kernel stack, dispatches on `v0`, returns results in `v0`/`a3`, and advances `EPC`.
- Exceptions are synchronous, interrupts asynchronous; both use the same entry path on MIPS.
- Always validate user pointers with `copyin`/`copyout`.

Chapter 5

Virtual Memory

5.1 Physical vs virtual addresses

- **Physical memory**: if the machine has P bytes of RAM, physical addresses range over $0 \ldots P-1$. A machine with 32-bit physical addresses can address at most 2^{32} bytes = 4 GB.
- **Virtual memory**: the kernel gives each process its own **address space**, a range of **virtual addresses** $0 \ldots 2^V - 1$. All addresses that a process generates (instruction fetches, loads, stores) are virtual.
- The **MMU** (memory management unit) translates every virtual address to a physical address, using information set up by the kernel.

Why virtualize memory?

1. **Isolation/protection**: a process can only reach physical memory that the kernel mapped into its address space.
2. **Convenience**: every program can be linked to start at the same virtual address; the address space can be larger than physical memory.
3. **Efficient use of memory**: only the parts actually used need physical memory (and pages can live on disk).

5.2 Dynamic relocation (base and bound)

The simplest scheme: each address space occupies one contiguous region of physical memory. The MMU has a **relocation register** R (base) and a **limit register** L (size).

$$\text{translate}(v) = \begin{cases} R + v & \text{if } v < L, \\ \text{exception} & \text{otherwise.} \end{cases}$$

On a context switch the kernel reloads R and L.

Example 5.1. $R =$ `0x24000`, $L =$ `0x10000`. Virtual `0x0F2A` $\to$ `0x24F2A`. Virtual `0x10004` $\geq L \to$ exception.

Problem: the whole address space (including the big unused gap between heap and stack) must be physically contiguous. This causes **external fragmentation**: free physical memory is split into holes too small to use.

5.3 Segmentation

Divide the address space into a few **segments** (code, data, heap, stack), each relocated independently with its own base and limit. A virtual address is (segment number, offset). With K bits for the segment number:

$$v = \underbrace{s}_{\text{top } K \text{ bits}} \mid \underbrace{o}_{\text{offset}}, \qquad \text{translate}(v) = R_s + o \text{ if } o < L_s.$$

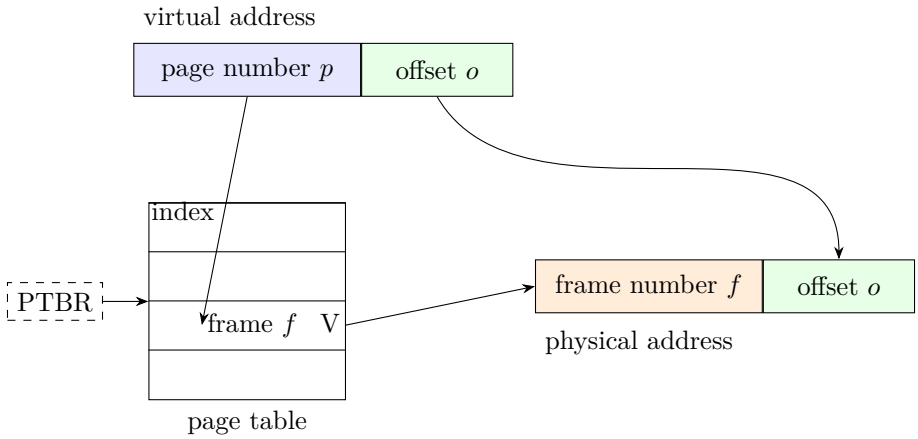
Unused gaps between segments need no physical memory, but each segment is still contiguous, so external fragmentation remains.

Example 5.2. 32-bit virtual addresses, 2-bit segment number. Segment 1 has base `0x40000` and limit `0x8000`. Virtual `0x40001F00` has segment $= 01_2 = 1$, offset `0x1F00` $<$ limit, so physical $=$ `0x41F00`.

5.4 Paging

Definition 5.3 (Paging)**.** Physical memory is divided into fixed-size **frames**; each virtual address space is divided into **pages** of the same size (typically 4 KB). Any page can be placed in any frame. A per-process **page table** maps page numbers to frame numbers.

A virtual address with page size 2^d splits into a **page number** (high bits) and an **offset** (low d bits). The offset is unchanged by translation.



Each **page table entry (PTE)** contains the frame number plus flag bits:

Valid the page is mapped; access to an invalid page raises an exception.

Write / read-only protection (e.g. code pages read-only); also execute bits.

Referenced (use) set by hardware on access; used by page replacement.

Dirty (modified) set on write; a dirty page must be written back before eviction.

Present the page is in memory (vs. on disk) — see page faults.

Example 5.4 (Translation)**.** 16-bit virtual addresses, 4 KB (2^{12}) pages: 4-bit page number, 12-bit offset. Page table: page 0 → frame `0x0F`, page 1 → frame `0x26`, page 2 invalid, page 3 → frame `0x01`.

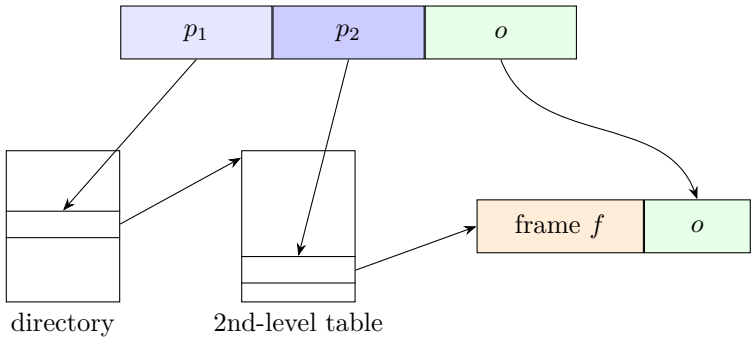
- `0x102C`: page 1, offset `0x02C` → `0x2602C`.
- `0x3FFF`: page 3, offset `0xFFF` → `0x01FFF`.
- `0x2004`: page 2 invalid → exception.

Paging pros and cons

No external fragmentation (all frames are interchangeable). Some **internal fragmentation**: on average half a page is wasted at the end of each segment. Main cost: the page table itself. With 32-bit addresses and 4 KB pages there are 2^{20} pages; with 4-byte PTEs a *linear* page table is 4 MB *per process*, even if the process uses little memory. With 48-bit or 64-bit virtual addresses, linear tables are impossible.

5.5 Multi-level page tables

Make the page table itself paged: split the page number into several pieces. The first piece indexes a top-level **directory**; each directory entry points to a second-level page table (or is invalid, in which case that whole region needs no table at all).



Example 5.5 (Two-level table for 32-bit addresses)**.** 4 KB pages ⇒ 12-bit offset, 20-bit page number split $10 + 10$. Each table has 2^{10} 4-byte entries = exactly one 4 KB page. A small process with one code page, one data page and one stack page near the top needs the directory plus 2 second-level tables (≈12 KB), instead of 4 MB.

General rule: choose the bits per level so each table fits in one page. With 2^d-byte pages and 2^e-byte PTEs, each level resolves $d - e$ bits, so the number of levels is $\lceil (V - d)/(d - e) \rceil$. E.g. x86-64: $V = 48$, $d = 12$, $e = 3 \Rightarrow \lceil 36/9 \rceil = 4$ levels.

Cost: a k-level table needs k memory accesses to translate one address. This is why a TLB is essential.

5.6 Translation lookaside buffer (TLB)

Definition 5.6 (TLB)**.** A **TLB** is a small, fast, fully-associative cache in the MMU holding recent (page number → frame number, flags) translations. On a TLB **hit**, translation takes no extra

memory access. On a **miss**, the translation must be found in the page table and loaded into the TLB.

- **Hardware-managed TLB** (x86): on a miss the MMU walks the page table itself; the format of the page table is fixed by the hardware.
- **Software-managed TLB** (MIPS): on a miss the MMU raises an exception; the kernel's handler finds the translation (any data structure it likes) and writes it into the TLB with `tlbwr` (random slot) or `tlbwi` (given index).

Effective access time with hit rate h, memory access time m, TLB lookup negligible, and a k-level table walk on a miss:

$$\text{EAT} = h \cdot m + (1 - h)(k + 1)m.$$

E.g. $h = 0.99$, $k = 2$: EAT $= 0.99m + 0.03m = 1.02m$. Programs have *locality*, so hit rates are typically very high.

On a context switch, TLB entries of the old process are wrong for the new one: either **flush** (invalidate) the TLB, or tag each entry with an **address space ID** (ASID). The MIPS R3000 TLB has 64 entries, each with a VPN in `EntryHi` (with a 6-bit ASID that OS/161 ignores) and PFN plus *valid*, *dirty* (= writable), *global*, *nocache* bits in `EntryLo`. OS/161's `as_activate` simply invalidates all TLB entries.

5.7 OS/161 dumbvm

OS/161 starts with `dumbvm`, a deliberately simple VM system:

- Each address space has exactly two segments (text and data) and a fixed-size stack (12 pages), each *contiguous in physical memory*: essentially segmentation with base and size stored in `struct addrspace` (`as_vbase1`, `as_pbase1`, `as_npages1`, ...).
- Physical memory is grabbed with `ram_stealmem` and never freed.
- On a TLB miss, `vm_fault` computes the physical address by relocation and writes a TLB entry.

```
int vm_fault(int faulttype, vaddr_t faultaddress) {
    struct addrspace *as = proc_getas(curproc);
    vaddr_t vbase1 = as->as_vbase1, vtop1 = vbase1 + as->as_npages1 * PAGE_SIZE;
    vaddr_t vbase2 = as->as_vbase2, vtop2 = vbase2 + as->as_npages2 * PAGE_SIZE;
    vaddr_t stackbase = USERSTACK - DUMBVM_STACKPAGES * PAGE_SIZE;
    paddr_t paddr;

    faultaddress &= PAGE_FRAME;              /* page-align */
    if (faulttype == VM_FAULT_READONLY) return EFAULT;  /* A3: kill process */
    if (faultaddress >= vbase1 && faultaddress < vtop1)
        paddr = faultaddress - vbase1 + as->as_pbase1;
    else if (faultaddress >= vbase2 && faultaddress < vtop2)
        paddr = faultaddress - vbase2 + as->as_pbase2;
    else if (faultaddress >= stackbase && faultaddress < USERSTACK)
        paddr = faultaddress - stackbase + as->as_stackpbase;
    else
        return EFAULT;                       /* segmentation fault */
```

```
    int spl = splhigh();
    uint32_t ehi = faultaddress;
    uint32_t elo = paddr | TLBLO_DIRTY | TLBLO_VALID;
    tlb_random(ehi, elo);                    /* A3: replace a random entry when full */
    splx(spl);
    return 0;
}
```

Assignment 3 improves this: make the text segment read-only after loading (clear `TLBLO_DIRTY`), handle a full TLB, and replace `ram_stealmem` with a **coremap** that tracks which physical frames are free so memory can be reclaimed.

5.8 Where address spaces come from

The kernel builds an address space from an ELF executable: the ELF header lists *segments* (virtual address, size in file, size in memory, permissions). For each, the kernel defines a region (`as_define_region`), allocates memory (`as_prepare_load`), copies the file image in, zero-fills the rest (`.bss`), and finally sets up the stack (`as_define_stack`). In OS/161 user stacks grow down from `USERSTACK = 0x80000000`.

5.9 Paging to secondary storage

Physical memory can be smaller than the sum of all address spaces. Keep only some pages in RAM; the rest live in a **swap** area on disk. The PTE *present* bit says which.

Definition 5.7 (Page fault)**.** A **page fault** is the exception raised when a process accesses a valid page that is not present in memory. The kernel:

1. finds a free frame, or evicts a victim page (writing it to disk if dirty, and marking its PTE not present);
2. reads the needed page from disk into the frame (the faulting thread blocks; another thread runs meanwhile);
3. updates the PTE (present, frame number) and returns, re-executing the faulting instruction.

Demand paging: pages are loaded only when first touched. Prefetching (read nearby pages together) can reduce fault counts.

Cost

A disk access is ∼5–10 ms vs. ∼100 ns for memory: a factor of 10^5. With fault rate p, EAT $= (1-p)\cdot 100\,\text{ns} + p\cdot 10\,\text{ms}$. To keep the slowdown under 10%, p must be below about 10^{-6}. Replacement policy matters.

5.10 Page replacement policies

Measured with a **reference string** (sequence of page numbers) and a number of frames.

5.10.1 Optimal (MIN / Belady)

Evict the page whose next use is furthest in the future. Provably minimal faults but requires knowledge of the future; used as a benchmark.

5.10.2 FIFO

Evict the page that has been in memory longest. Simple but ignores usage.

Example 5.8 (3 frames, reference string `a b c d a b e a b c d e`).

Ref	a	b	c	d	a	b	e	a	b	c	d	e
FIFO F1	a	a	a	d	d	d	e	e	e	e	e	e
FIFO F2		b	b	b	a	a	a	a	a	c	c	c
FIFO F3			c	c	c	b	b	b	b	b	d	d
Fault	x	x	x	x	x	x	x			x	x	

FIFO: 9 faults. OPT on the same string: 7 faults (a, b, c, d, e, c, d). With 4 frames FIFO gives 10 faults — more frames, more faults! This is **Belady's anomaly**.

5.10.3 LRU (least recently used)

Evict the page whose most recent use is furthest in the past — past behaviour predicts the future (locality). LRU is a *stack algorithm* and does not suffer Belady's anomaly. On the string above with 3 frames LRU gives 10 faults (this string is adversarial for it). Exact LRU requires updating a timestamp or list on *every* memory reference, which is too expensive; hardware only provides the use bit.

5.10.4 Clock (second chance)

Approximates LRU with the *use bit*. Frames are arranged in a circle with a "hand".

```
int clock_choose_victim(void) {
    for (;;) {
        struct frame *f = &frames[hand];
        hand = (hand + 1) % nframes;
        if (f->use) f->use = 0;     /* give a second chance */
        else return f - frames;     /* not used since last sweep: evict */
    }
}
```

Hardware sets `use = 1` on every access. On a MIPS software-managed TLB, the kernel can simulate use bits by invalidating TLB entries and setting the bit when the next TLB miss for that page occurs. A common refinement also prefers clean pages (use bit, dirty bit) over dirty ones.

5.10.5 Working sets and thrashing

Definition 5.9. The **working set** $W(t, \Delta)$ of a process is the set of pages it referenced in the last Δ time units (or references). **Thrashing** occurs when the sum of working sets exceeds physical memory: processes fault continually, the CPU is mostly idle waiting for the disk, and throughput collapses.

Remedies: give each process enough frames for its working set (*page-fault frequency* control: if a process faults too often give it more frames, if rarely take some away), and if memory is overcommitted, suspend (swap out) whole processes to reduce the *multiprogramming level*.

> Locality: *temporal* (recently used data is used again soon) and *spatial* (nearby addresses are used soon). Both are what make TLBs, caches and LRU-like replacement effective.

Summary

- The MMU translates virtual to physical addresses on every access; the kernel sets up the mapping and handles exceptions.
- Base/bound and segmentation are simple but suffer external fragmentation; paging avoids it at the cost of page tables (internal fragmentation is small).
- Multi-level page tables save space for sparse address spaces; TLBs make translation fast. MIPS uses a software-managed 64-entry TLB.
- `dumbvm` = contiguous segments + TLB refill by relocation.
- Page faults bring pages in from disk; replacement policies: OPT (ideal), FIFO (Belady's anomaly), LRU (good, expensive), clock (practical approximation).
- Thrashing occurs when working sets do not fit in memory.

Chapter 6

Scheduling

6.1 The scheduling problem

The scheduler decides which ready thread runs next on each CPU. We start with a simplified model of **jobs**: each job j has an **arrival time** a_j and a **run time** r_j, and is CPU-bound (no blocking). Metrics for a schedule:

Turnaround time $t_j - a_j$, where t_j is the completion time.

Response time $s_j - a_j$, where s_j is when the job first runs.

Throughput jobs completed per unit time.

Fairness each job gets a reasonable share; no **starvation** (a job never runs).

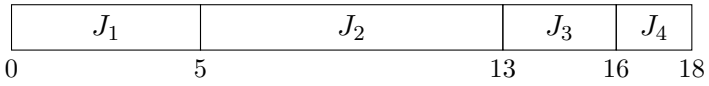
Usually we minimize *average* turnaround or response time. Interactive systems care about response time; batch systems about turnaround and throughput.

Running example used below:

Job	Arrival	Run time
J_1	0	5
J_2	0	8
J_3	0	3
J_4	5	2

6.2 First come, first served (FCFS/FIFO)

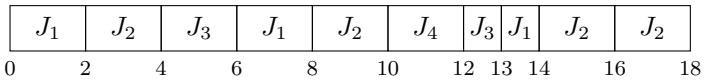
Run jobs in arrival order to completion (ties in index order). Non-preemptive.



Turnaround: $5, 13, 16, 13 \Rightarrow$ average $47/4 = 11.75$. Short jobs stuck behind long ones (**convoy effect**).

6.3 Round robin (RR)

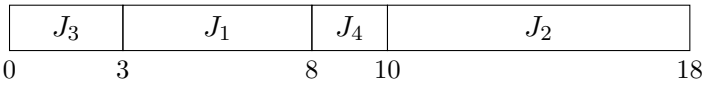
Preemptive FCFS: each job runs for at most a quantum q, then goes to the back of the ready queue. With $q = 2$ (a job arriving at time t is enqueued before a job preempted at t):



Turnaround: $J_1 = 14$, $J_2 = 18$, $J_3 = 13$, $J_4 = 12 - 5 = 7 \Rightarrow$ average 13. Response times are much better ($0, 2, 4, 5$ vs. FCFS $0, 5, 13, 11$). RR trades turnaround for response time. Quantum choice: too small $\Rightarrow$ context-switch overhead; too large $\Rightarrow$ degenerates into FCFS.

6.4 Shortest job first (SJF)

Non-preemptive: when the CPU becomes free, run the ready job with the smallest run time.



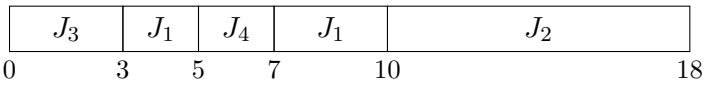
Turnaround: $J_3 = 3$, $J_1 = 8$, $J_4 = 5$, $J_2 = 18 \Rightarrow$ average $34/4 = 8.5$.

Theorem 6.1. *If all jobs arrive at the same time, SJF minimizes average turnaround time.*

Sketch. Running jobs in order $r_{(1)} \le \cdots \le r_{(n)}$, the total turnaround is $\sum_k (n-k+1) r_{(k)}$: the k-th job delays itself and all later jobs. An exchange argument shows any adjacent pair out of order can be swapped to reduce the sum. □

6.5 Shortest remaining time first (SRTF)

Preemptive SJF: when a job arrives, if its run time is less than the remaining time of the running job, preempt. In the example, at $t = 5$, J_1 (started at 3) has 3 remaining and J_4 needs 2, so J_4 preempts:



Turnaround: $3, 10, 2, 18 \Rightarrow$ average $33/4 = 8.25$.

SJF/SRTF in practice

They are optimal for turnaround but (1) need to know run times in advance and (2) can **starve** long jobs if short jobs keep arriving. Real schedulers estimate the next CPU burst from history, e.g. exponential averaging $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$.

6.6 Real threads: CPU bursts and I/O

Real threads alternate CPU *bursts* with blocking for I/O. **Interactive** (I/O-bound) threads have short bursts and need good response time; **CPU-bound** threads have long bursts. Good schedulers favour I/O-bound threads: they quickly issue I/O and block, keeping devices busy while CPU-bound work fills the gaps. Many systems also use **priorities**; strict priority scheduling can starve low-priority threads, so priorities are often adjusted dynamically (*aging*).

6.7 Multi-level feedback queue (MLFQ)

MLFQ approximates SJF without knowing run times, by learning from behaviour.

- There are n round-robin queues Q_n (highest priority) $\dots Q_1$ (lowest); lower-priority queues often have longer quanta.
- Always run a thread from the highest non-empty queue.
- A new thread enters the top queue.
- If a thread uses its whole quantum (CPU-bound), it is **demoted** one level.
- If a thread blocks before its quantum expires (interactive), it stays (or is promoted).
- To prevent starvation and adapt to phase changes, periodically **boost** all threads back to the top queue.

Gaming: a thread that yields just before its quantum ends could stay on top forever; a fix is to charge accumulated CPU time at each level, not just per quantum. MLFQ-style schedulers were used in BSD Unix, Windows NT and Solaris.

6.8 Linux Completely Fair Scheduler (CFS)

CFS aims to give each thread a share of the CPU proportional to its **weight**, which is derived from its *nice* value (lower nice, higher weight; each nice step is about a 1.25× change).

- Each thread tracks its **virtual runtime**:

  $$\text{vruntime} \mathrel{+}= \text{actual runtime} \times \frac{w_0}{w_i},$$

  where w_0 is the weight of nice level 0. Higher-weight threads accumulate vruntime more slowly.
- The scheduler always runs the thread with the *smallest* vruntime; ready threads are kept in a red-black tree ordered by vruntime ($\mathcal{O}(\log n)$ insert, $\mathcal{O}(1)$ leftmost lookup with caching).
- The time slice is computed from a target latency (e.g. 20 ms, the period in which every ready thread should run once) divided in proportion to weights, with a minimum granularity so slices do not become tiny.
- A thread that wakes after sleeping gets its vruntime set close to the current minimum, so it runs soon (good response) but cannot claim credit for the whole time it slept.

Example 6.2. Two threads with weights 2 and 1 (sum 3), target latency 30 ms: they get slices of 20 ms and 10 ms. After both run once, actual runtime 20 and 10, vruntime increments $20 \cdot w_0/2$ and $10 \cdot w_0/1$ — equal when $w_0 = 1$: both are "fair".

6.9 Multicore scheduling

Single shared ready queue Any core takes the next thread from one global queue. Simple and naturally load balanced, but the queue lock is a scalability bottleneck, and threads hop between cores, losing **cache affinity**.

Per-core ready queues Each core schedules from its own queue. Scales well, and a thread keeps running on the same core, so its data stays warm in that core's caches (and TLB). Problem: **load imbalance** (one core idle while another has many threads). Fix: *load balancing* by migrating threads — push migration (a periodic balancer moves threads) or **work stealing**/pull migration (an idle core steals from a busy one). Migration costs cache warmth, so it is done sparingly.

Cache affinity: after a thread runs on core c, its working set is in c's L1/L2 cache. Moving it to another core means refetching that data from memory (or a shared L3), which can cost much more than a context switch itself. Linux CFS uses per-core run queues with periodic balancing across a hierarchy of *scheduling domains* (hyperthreads, cores, sockets/NUMA nodes).

Summary

Policy	Preemptive?	Notes
FCFS	no	simple, convoy effect
RR	yes	good response; quantum trade-off
SJF	no	optimal avg turnaround (same arrivals); needs run times; starvation
SRTF	yes	optimal avg turnaround; starvation
MLFQ	yes	learns burst behaviour; priority boost avoids starvation
CFS	yes	weighted fair share via vruntime and a red-black tree

Multicore: per-core queues for scalability and cache affinity, plus load balancing.

Chapter 7

Devices and I/O

7.1 Devices and buses

Devices (disks, network cards, timers, serial consoles, GPUs) connect to the CPU and memory through one or more **buses**. A typical system has a fast memory bus, an I/O bus (PCI Express), and slower peripheral buses (USB, SATA). In System/161 all devices sit on a simple `LAMEbus` with up to 32 slots.

Definition 7.1 (Device driver)**.** A **device driver** is the part of the kernel that interacts with a particular device: it knows the device's registers and protocol, and presents a uniform interface (e.g. "read block n") to the rest of the kernel.

7.2 Device registers

The kernel communicates with a device through **device registers**, typically of three kinds:

Status read to learn the device's state (busy, done, error).

Command written to tell the device what to do.

Data used to transfer data to/from the device.

Two ways to access device registers:

- **Port-mapped I/O**: special instructions (`in`/`out` on x86) address a separate I/O address space.
- **Memory-mapped I/O**: registers appear at physical addresses; ordinary loads and stores access them. MIPS/System/161 uses this: each LAMEbus slot gets a 64 KB region of physical addresses starting at `0x1FE00000`, which the kernel accesses through the uncached segment `kseg1` (`0xA0000000` + physical) so that reads/writes really reach the device.

Example 7.2 (System/161 timer/clock device)**.**

Offset	Size	Type	Description
0	4	status	current time (seconds)
4	4	status	current time (nanoseconds)
8	4	command	restart-on-expiry flag
12	4	status and command	interrupt (reading acknowledges it)
16	4	command	countdown time (microseconds)
20	4	command	speaker (causes a beep)

Writing 1000000 to offset 16 with restart set makes the timer interrupt every second.

7.3 Polling vs interrupts

Example 7.3 (System/161 serial console, write one character)**.** Registers: *character buffer* (data), *write IRQ* (status/command), *read IRQ*.

Polling (busy waiting):

```
void serial_putc_polled(char c) {
    write_reg(SERIAL_CHAR, c);            /* start the operation */
    while (read_reg(SERIAL_WIRQ) == 0)    /* spin until device says done */
        ;
    write_reg(SERIAL_WIRQ, 0);            /* acknowledge completion */
}
```

Interrupt-driven:

```
void serial_putc(char c) {
    P(serial_write_sem);                  /* only one write in progress (init 1) */
    write_reg(SERIAL_CHAR, c);            /* start; return immediately */
}
void serial_interrupt_handler(void) {     /* device raises an interrupt when done */
    write_reg(SERIAL_WIRQ, 0);            /* acknowledge */
    V(serial_write_sem);                  /* allow the next write */
}
```

Trade-off

Polling wastes CPU while the device works, but has very low latency and no interrupt overhead; it is fine for very fast devices or when the CPU has nothing else to do. Interrupts let the CPU run other threads while the device works, at the cost of interrupt handling overhead per event. Very high-rate devices (fast network cards) switch adaptively between the two (e.g. Linux NAPI).

7.4 Moving data: programmed I/O vs DMA

Programmed I/O (PIO) The CPU itself copies data between memory and the device's data registers or on-device buffer, word by word. Simple, but the CPU is busy for the whole transfer.

Direct memory access (DMA) The device transfers data directly to/from main memory. The driver tells the device the physical memory address, the length and the operation; the device

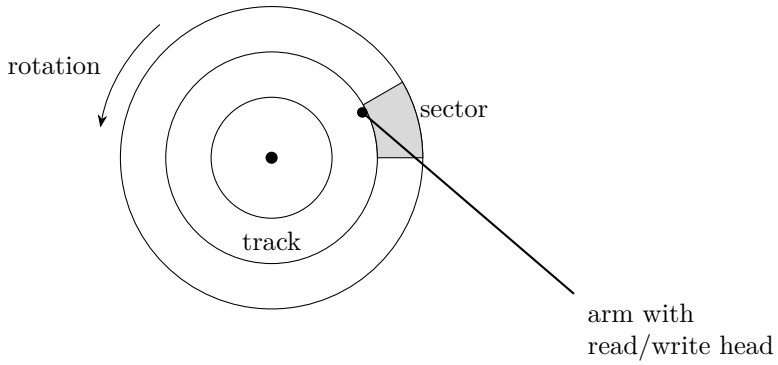
then masters the bus, copies the data, and raises an interrupt on completion. The CPU is free during the transfer (it competes with the device for the memory bus only).

```c
/* DMA disk read, conceptually */
void disk_read_dma(uint32_t sector, paddr_t buf, size_t nsectors) {
    lock_acquire(disk_lock);
    write_reg(DISK_SECTOR, sector);
    write_reg(DISK_ADDR,   buf);          /* PHYSICAL address of the buffer */
    write_reg(DISK_COUNT,  nsectors);
    write_reg(DISK_CMD,    CMD_READ);     /* start */
    P(disk_done);                         /* block; interrupt handler does V */
    lock_release(disk_lock);
}
```

Remark. DMA uses physical addresses, so the buffer must be contiguous in physical memory (or the device must support scatter/gather lists) and must not be paged out during the transfer (*pinned*).

System/161 disks use PIO through a 512-byte on-device transfer buffer that is memory-mapped.

7.5 Hard disk drives



A disk has one or more **platters** rotating at constant speed (5400–15000 RPM). Each surface is divided into concentric **tracks**, each track into **sectors** (512 B or 4 KB). Tracks at the same radius on all surfaces form a **cylinder**. An **arm** moves the read/write heads radially. The drive presents a linear array of logical block numbers; nearby block numbers are physically nearby.

Cost of an I/O request of k sectors:

$$T = T_{\text{seek}} + T_{\text{rot}} + T_{\text{transfer}}.$$

Seek time move the heads to the right track: from 0 (same track) up to a maximum of roughly 10–20 ms; a common model is linear in seek distance, average $\approx$ one third of the maximum for random requests.

Rotational latency wait for the sector to come under the head: between 0 and one full rotation ω^{-1}; average half a rotation.

Transfer time k/(sectors per track)$\times$ rotation time.

Example 7.4. 10000 RPM $\Rightarrow$ one rotation $= 60/10000\,\text{s} = 6\,\text{ms}$, average rotational latency 3 ms. Average seek 5 ms. 1000 sectors per track; reading 8 sectors (4 KB): transfer $= 8/1000 \times 6\,\text{ms} = 0.048\,\text{ms}$. Total $\approx 8.05\,\text{ms}$, so random 4 KB reads achieve about $4\,\text{KB}/8\,\text{ms} \approx 0.5\,\text{MB/s}$, while sequential reads approach $1000 \times 512\,\text{B}/6\,\text{ms} \approx 85\,\text{MB/s}$.

Moral

Seek and rotation dominate. Large, sequential transfers are vastly faster than small random ones. File systems and disk schedulers try to make I/O sequential and to reduce seek distances.

7.6 Disk head scheduling

When several requests are pending, the order matters. Example: head at cylinder 53 (moving toward higher numbers), queue $98, 183, 37, 122, 14, 124, 65, 67$, cylinders 0..199.

FCFS Serve in arrival order. Fair but poor: $53 \to 98 \to 183 \to 37 \to 122 \to 14 \to 124 \to 65 \to 67$, total movement $45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640$ cylinders.

SSTF (shortest seek time first) Always serve the closest request: $53 \to 65 \to 67 \to 37 \to 14 \to 98 \to 122 \to 124 \to 183$, total $12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236$. Greedy and good, but can **starve** far requests.

SCAN (elevator) Move in one direction serving requests, reverse at the last request in that direction (strictly this variant is called LOOK): $53 \to 65 \to 67 \to 98 \to 122 \to 124 \to 183 \to 37 \to 14$, total $130 + 169 = 299$. No starvation; middle cylinders get better service.

C-SCAN Serve only while moving in one direction, then jump back to the start (circular). More uniform waiting times.

Modern drives do their own scheduling internally (they know the exact geometry and rotational position); the OS keeps queues deep enough for the drive to optimize, and may still merge and sort requests.

7.7 Solid state drives (SSDs)

SSDs use NAND flash: no moving parts, so no seek or rotational latency; random reads are fast ($\sim$tens of μs).

- Flash is organized into **pages** (e.g. 4 KB, the unit of read/write) grouped into **blocks** (e.g. 128 pages, the unit of erase).
- A page cannot be overwritten in place: it must be erased first, and erase works only on whole blocks and is slow (ms).
- Each block survives a limited number of program/erase cycles (**wear out**).
- The **flash translation layer** (FTL) in the drive maps logical blocks to physical pages. Writes go to fresh erased pages (log-structured style); old copies are marked invalid; **garbage collection** copies valid pages out of mostly-invalid blocks and erases them; **wear levelling** spreads erases evenly.

- The OS can help with **TRIM**: tell the drive which logical blocks are no longer in use. Disk head scheduling is irrelevant for SSDs; defragmentation is harmful (extra writes).

7.8 Other persistent storage

Persistent memory (e.g. byte-addressable NVM) blurs the line between memory and storage; RAID combines multiple disks for performance (striping) and reliability (mirroring, parity).

Summary

- Drivers talk to devices through status/command/data registers, via port-mapped or memory-mapped I/O (System/161: memory-mapped, accessed through uncached kseg1).
- Completion is detected by polling or interrupts; data is moved by programmed I/O or DMA.
- Disk access time = seek + rotational latency + transfer; seek and rotation dominate.
- Disk scheduling: FCFS (fair, slow), SSTF (greedy, starvation), SCAN/elevator and C-SCAN.
- SSDs: no mechanical latency, but erase-before-write in blocks, wear; the FTL hides this.

Chapter 8

File Systems

8.1 Files and file systems

Definition 8.1. A **file** is a persistent, named sequence of bytes with associated **metadata** (size, owner, permissions, timestamps). A **file system** is the data structure on a storage device (and the kernel code) that implements files and directories on top of fixed-size blocks.

The OS provides a logical view (files, directories, byte offsets) and maps it to the physical view (block numbers on a device).

8.2 File interface

```c
int fd = open("/home/vg/notes.txt", O_RDWR | O_CREAT, 0644);
                                    /* returns a file descriptor */
char buf[512];
ssize_t n = read(fd, buf, sizeof buf); /* reads from the current offset */
lseek(fd, 0, SEEK_SET);               /* reposition the offset */
write(fd, "hello", 5);                /* writes at the current offset */
fsync(fd);                            /* force data to stable storage */
close(fd);
```

- A **file descriptor** is a small integer indexing the process's file descriptor table. Each entry points to an *open file* object in the kernel holding the current **offset** and access mode, which points to the file's in-memory i-node (vnode).
- After `fork`, parent and child share open file objects (and therefore offsets).
- Reads/writes are *sequential* by default (offset advances); `lseek` enables random access. Some APIs (`pread`/`pwrite`) take an explicit offset.
- `write` usually only updates the in-memory buffer cache; data reaches the disk later unless `fsync` is called.

8.3 Directories and links

Files are organized in a hierarchical namespace: a tree of **directories**. A directory is a special file containing **entries** mapping names to i-node numbers. A **path** (`/home/vg/notes.txt`) is resolved

component by component from the root (absolute) or the current working directory (relative). `.` is the directory itself, `..` its parent.

Definition 8.2 (Hard link)**.** A **hard link** is a directory entry associating a name with an i-node. A file may have several hard links (in different directories or with different names); all are equally "the file". `link(old, new)` adds one, `unlink(name)` removes one. The i-node keeps a **link count**; the file's data is freed only when the count reaches 0 (and no process has it open).

Restrictions: hard links cannot cross file systems (i-node numbers are per file system), and users may not hard-link directories (to keep the directory structure a tree / DAG without cycles).

Definition 8.3 (Symbolic (soft) link)**.** A **symbolic link** is a special file whose content is a path name. When path resolution reaches it, the kernel substitutes the stored path. Created with `symlink(target, name)`.

Symbolic links can cross file systems and point to directories, but they can *dangle*: if the target is removed, the link points to nothing. The target's link count does not change.

Example 8.4.

```
echo hi > a       # i-node 42, link count 1
ln a b            # hard link: b -> i-node 42, link count 2
ln -s a c         # c is a new i-node whose content is "a"
rm a              # link count 1; b still reads "hi"; c now dangles
```

Multiple file systems

Each device/partition holds its own file system. Windows uses drive letters; Unix **mounts** a file system at a directory of the existing namespace (`mount /dev/sdb1 /mnt/usb`), creating one global tree. OS/161's VFS uses device names (`lhd0:`).

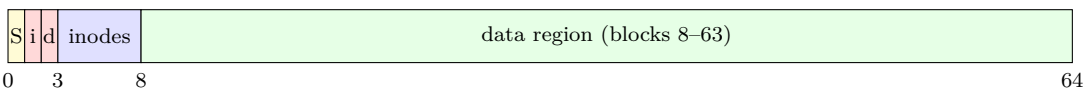
8.4 File system implementation

What must be stored on disk:

- file data;
- file metadata (i-nodes);
- directories (names → i-node numbers);
- free-space information for data blocks and i-nodes;
- file-system-wide information (the **superblock**: size, number of i-nodes, location of the tables, magic number).

8.4.1 Very Simple File System (VSFS) layout

Example from OSTEP: a 256 KB disk with 4 KB blocks = 64 blocks.



- Block 0: superblock.
- Block 1: i-node bitmap (1 bit per i-node: free/used).
- Block 2: data bitmap (1 bit per data block).
- Blocks 3–7: i-node table. With 256-byte i-nodes, $5 \times 4096/256 = 80$ i-nodes.
- Blocks 8–63: data blocks.

To find i-node number k: block $= 3 + \lfloor k \cdot 256/4096 \rfloor$, offset $= (k \cdot 256) \bmod 4096$.

8.4.2 I-nodes

An **i-node** (index node) is a fixed-size structure holding all metadata of a file *except its name*: type (file, directory, symlink), permissions, owner, size, link count, timestamps, and the locations of its data blocks.

Definition 8.5 (Indirect blocks)**.** To support small and large files with a fixed-size i-node, the i-node contains

- D **direct pointers** to data blocks;
- a **single indirect** pointer to a block full of pointers to data blocks;
- a **double indirect** pointer to a block of pointers to single-indirect blocks;
- a **triple indirect** pointer, and so on.

Example 8.6 (Maximum file size)**.** 4 KB blocks, 4-byte block pointers ($\Rightarrow P = 1024$ pointers per block), 12 direct pointers, one single, one double, one triple indirect:

$$\text{max blocks} = 12 + 1024 + 1024^2 + 1024^3 \approx 2^{30}, \qquad \text{max size} \approx 2^{30} \times 4\,\text{KB} = 4\,\text{TB}.$$

Reading byte offset 10 000 000: block index $\lfloor 10^7/4096 \rfloor = 2441$. Since $2441 \geq 12 + 1024 = 1036$, it is in the double-indirect range at index $2441 - 1036 = 1405$: first-level slot $\lfloor 1405/1024 \rfloor = 1$, second-level slot $1405 \bmod 1024 = 381$. Locating it needs 3 block reads (double-indirect block, single-indirect block, data) after the i-node.

Alternatives: **extents** (start block, length) pairs, used by ext4/NTFS, describe large contiguous files compactly; FAT uses a linked list of blocks stored in a table.

8.4.3 Directories on disk

A directory's data blocks hold entries (i-node number, record length, name length, name). Looking up a name scans the entries (or uses a hash/B-tree index in modern file systems). Deleting an entry can leave a gap that is merged into the previous entry's record length.

8.4.4 Example: reading `/foo/bar`

1. Read the root i-node (number well-known, e.g. 2), then root's data block(s) to find `foo`'s i-node number.
2. Read `foo`'s i-node, then its data to find `bar`'s i-node number.
3. Read `bar`'s i-node (permission check, `open` returns fd).
4. Each `read()` of a block: read the i-node (block locations), read the data block, *write* the i-node (update access time).

Creating a file is worse: read/write the i-node bitmap, write the new i-node, write the directory data and the directory's i-node, plus data bitmap and data block writes when data is written. The **buffer cache** (caching recent blocks in memory) and write-back/delayed writes absorb most of this traffic.

8.4.5 Block allocation

Free blocks/i-nodes are tracked with bitmaps (or free lists). Allocation policy matters for performance on HDDs: keep a file's blocks contiguous, and keep files of the same directory close to each other and to their i-nodes (e.g. the Fast File System divides the disk into *cylinder groups*, each with its own bitmaps, i-nodes and data). Some file systems pre-allocate several blocks at a time to reduce fragmentation.

8.5 Crash consistency

Updating a file requires several separate disk writes. Example: appending one block to a file requires writing (1) the data bitmap, (2) the i-node (new pointer, new size), and (3) the data block. A crash (power loss) can occur between any two writes.

Writes completed	Result
data only	no harm (data lost, but FS consistent)
i-node only	i-node points to garbage; bitmap says block free (*inconsistent*)
bitmap only	space leak: block marked used but no file owns it
i-node + bitmap	FS consistent, but file contains garbage
i-node + data	inconsistent bitmap (block could be reallocated)
bitmap + data	space leak

8.5.1 fsck

After an unclean shutdown, a file system checker scans everything: superblock sanity, rebuild the bitmaps from i-nodes, check link counts against directory entries, find duplicate or bad block pointers, put orphaned i-nodes in `lost+found`. Correct but very slow (time proportional to disk size, not to the amount of recent work), and it cannot fix "consistent but garbage" cases.

8.5.2 Journaling (write-ahead logging)

Before updating structures in place, first write a description of the update to a **journal** (log) area.

1. **Journal write**: write a transaction-begin block (TxB, with a transaction ID), then the blocks to be written (e.g. new bitmap, i-node, data) to the log.
2. **Journal commit**: once those are on disk, write a transaction-end block (TxE, a single sector so it is written atomically). The transaction is now durable.
3. **Checkpoint**: write the blocks to their real locations.
4. **Free**: mark the transaction as done in the journal superblock, so the space can be reused.

Recovery: scan the log; **replay** (redo) every committed transaction (TxB...TxE complete); discard incomplete ones. Replaying twice is harmless (redo logging is idempotent). Recovery time is proportional to the log size, not the disk size.

Metadata (ordered) journaling

Data journaling writes every data block twice. Most systems (ext3/ext4 default "ordered" mode, NTFS) journal *only metadata*: write the data block to its final location *first*, then journal and commit the metadata, then checkpoint. This rule "write the pointed-to object before the pointer" guarantees an i-node never points to garbage. Checksums over the transaction let TxE be written together with the other blocks.

Other approaches: **soft updates** (carefully order all writes so the on-disk state is always consistent), **copy-on-write** file systems (ZFS, btrfs: never overwrite; write new versions and atomically switch the root pointer), and **log-structured** file systems (the whole disk is a log).

8.6 OS/161 VFS and SFS

OS/161 has a virtual file system layer (`vfs_open`, `VOP_READ`, `VOP_WRITE` on `struct vnode`) that hides differences between file systems: `emufs` (passes calls to the host file system) and `SFS`, a simple Unix-like file system with a superblock, a free-block bitmap, i-nodes with direct and single-indirect pointers, and directories of fixed-size entries. Data moves between kernel and user space with `struct uio`.

Summary

- Files: named, persistent byte sequences with metadata; accessed through file descriptors and per-open offsets.
- Directories map names to i-node numbers. Hard links share an i-node (link count); symbolic links store a path and can dangle.
- On disk: superblock, i-node and data bitmaps, i-node table, data blocks (VSFS).
- I-nodes locate data with direct, single, double, triple indirect pointers.
- Multi-write updates are vulnerable to crashes; fsck repairs slowly, journaling commits updates atomically to a log and replays committed transactions after a crash.

Chapter 9

Virtual Machines

This module was added in Fall 2020 (listed as "New!" on that term's reading page). In Winter 2018 the final slot was "Interprocess Communication and Networking", marked as not covered, so networking is only mentioned briefly at the end.

9.1 What is a virtual machine?

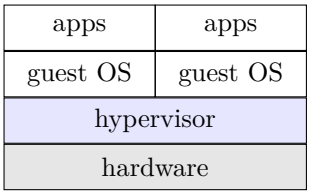
Definition 9.1. A **virtual machine** (VM) is a software abstraction of a complete computer: CPU, memory, devices. A **guest** operating system runs inside the VM. The **hypervisor** (or *virtual machine monitor*, VMM) is the software that creates and runs VMs on the real **host** hardware, multiplexing it among the guests.

Just as a process virtualizes a CPU and memory for an application, a VM virtualizes the whole machine for an operating system.

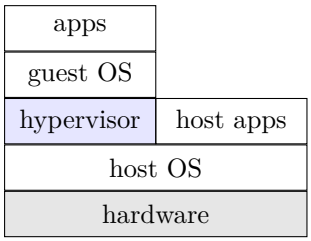
Uses:

- **Server consolidation**: many lightly loaded servers on one physical machine; the basis of cloud computing (AWS EC2, etc.).
- **Isolation**: a compromised or crashed guest does not affect others.
- Running different OSes side by side (development, testing, legacy software).
- **Snapshots and live migration**: VM state can be saved, cloned, or moved to another host while running.
- OS development and teaching — System/161 is itself a (simulated) machine!

9.2 Types of hypervisors



Type 1 (bare metal)



Type 2 (hosted)

Type 1 (native, bare metal) runs directly on the hardware: Xen, VMware ESXi, Microsoft Hyper-V. Often a small privileged guest (Xen's *dom0*) handles device drivers.

Type 2 (hosted) runs as an application on a host OS and uses the host's drivers: VMware Workstation, VirtualBox, QEMU. Linux KVM blurs the line (the Linux kernel itself becomes the hypervisor).

9.3 Virtualizing the CPU

9.3.1 Trap and emulate

Run the guest OS *directly* on the CPU, but in user mode (deprivileged). Ordinary instructions run at full speed. When the guest executes a privileged instruction (e.g. write the TLB, disable interrupts, return from exception), the CPU traps to the hypervisor, which **emulates** the effect on the VM's *virtual* state (virtual CPU registers, virtual interrupt flag, virtual TLB) and returns.

System calls and exceptions from guest applications also trap into the hypervisor first; it reflects them into the guest OS by jumping to the guest's exception handler as if the hardware had done so (and records that the virtual CPU is now in "virtual kernel mode").

Theorem 9.2 (Popek and Goldberg, 1974, informally)**.** *A machine can be virtualized by trap-and-emulate if every* sensitive *instruction (one that reads or changes privileged machine state or behaves differently in user vs. kernel mode) is also* privileged *(traps when executed in user mode).*

Classic x86 violated this: e.g. `popf` in user mode silently ignores changes to the interrupt flag instead of trapping. Solutions:

- **Binary translation** (VMware, 1999): scan guest kernel code before running it and rewrite problematic instructions into calls to the hypervisor; cache translated blocks.
- **Paravirtualization** (Xen): modify the guest OS to call the hypervisor explicitly (**hypercalls**) instead of using sensitive instructions. Fast, but requires guest source changes.
- **Hardware-assisted virtualization** (Intel VT-x, AMD-V, 2005+): a new CPU mode ("guest mode" / non-root mode) in which the guest kernel runs at its normal privilege level; specified events cause a *VM exit* to the hypervisor, which later does a *VM entry*. Guest state is kept in a control structure (VMCS).

9.4 Virtualizing memory

Now there are three kinds of addresses:

$$\text{guest virtual} \xrightarrow{\text{guest page table}} \text{guest physical} \xrightarrow{\text{hypervisor}} \text{host (machine) physical.}$$

The guest OS believes it controls physical memory; the hypervisor actually allocates machine frames to the VM.

Shadow page tables The hypervisor write-protects the guest's page tables; whenever the guest changes them, it traps, and the hypervisor updates a *shadow* page table that maps guest virtual directly to machine physical. The real MMU uses the shadow table. Correct but many expensive traps.

Software-managed TLB (MIPS) On a TLB miss the hypervisor's handler gets control, runs the guest's refill handler (whose `tlbwr` traps), translates the guest physical frame to the machine frame, and installs the combined mapping.

Nested / extended page tables (Intel EPT, AMD NPT) The hardware walks *both* tables: guest page table (guest virtual → guest physical) and the hypervisor's table (guest physical → machine). No traps on guest page table updates, but a TLB miss is expensive: with two 4-level tables a full walk can take up to 24 memory references ($(4+1)(4+1)-1$), so large pages and big TLBs matter.

Memory overcommitment: the hypervisor can page guest memory to disk, use a **balloon driver** inside the guest (the balloon "allocates" guest pages so the guest's own replacement policy chooses what to give up, and the hypervisor reclaims the backing frames), and share identical pages between VMs copy-on-write (**content-based page sharing**).

9.5 Virtualizing I/O

- **Emulation**: present a well-known virtual device (e.g. an old network card or IDE disk); the guest's ordinary driver accesses its registers, each access traps, and the hypervisor emulates the device using real host devices. Works with unmodified guests but slow.
- **Paravirtual devices** (virtio, Xen split drivers): the guest uses a special driver that communicates with the hypervisor through shared-memory ring buffers, batching requests and avoiding per-register traps.
- **Device passthrough**: give a VM direct access to a physical device (or a virtual function of an SR-IOV device), with an IOMMU translating and restricting the device's DMA addresses so it can only reach that VM's memory.

9.6 Containers (OS-level virtualization)

Containers (Docker, LXC) isolate groups of processes that *share one kernel*: namespaces give each container its own view of PIDs, file system mounts, network interfaces and users; cgroups limit their resource usage. Much lighter than VMs (no guest kernel), but weaker isolation and the same OS kernel for everyone.

9.7 Networking (brief, for context)

Not examined in the terms covered here, but for completeness: the OS implements the network protocol stack (link layer driver, IP routing, TCP/UDP) in the kernel and exposes it via the **socket** API (`socket`, `bind`, `listen`, `accept`, `connect`, `send`, `recv`). Sockets are also the main mechanism for inter-process communication between machines, complementing pipes and shared memory on a single machine.

```
int s = socket(AF_INET, SOCK_STREAM, 0);          /* TCP socket */
struct sockaddr_in addr = { .sin_family = AF_INET,
                            .sin_port = htons(8080),
                            .sin_addr.s_addr = htonl(INADDR_ANY) };
bind(s, (struct sockaddr *)&addr, sizeof addr);
listen(s, 16);
int c = accept(s, NULL, NULL);                    /* blocks for a client */
char buf[256]; ssize_t n = recv(c, buf, sizeof buf, 0);
send(c, buf, n, 0);                               /* echo it back */
close(c); close(s);
```

Summary

- A hypervisor multiplexes hardware among guest OSes; Type 1 runs on bare metal, Type 2 on a host OS.
- CPU: trap-and-emulate works when all sensitive instructions are privileged (Popek–Goldberg); otherwise binary translation, paravirtualization, or hardware support (VT-x/AMD-V).
- Memory: guest virtual → guest physical → machine physical, via shadow page tables or hardware nested page tables; ballooning and page sharing for overcommit.
- I/O: device emulation, paravirtual drivers, or passthrough with an IOMMU.
- Containers share the host kernel and are lighter but less isolated than VMs.